

Similaridade de Cosseno entre Embeddings

Aula 16 – Introdução ao Processamento de Linguagem Natural (PLN)

Eduardo Corrêa Gonçalves

14/05/2026

Sumário

Introdução

Similaridade entre Embeddings Semânticos

Similaridade de Cosseno

Exemplos na Spacy

Introdução (1/3)

- **Embeddings no Python: spaCy**
 - spaCy é uma dos mais importantes *toolkits* para PLN do Python
 - <https://spacy.io/>
 - Conforme visto na última aula, ela disponibiliza **embeddings Tok2Vec** para diversos idiomas, incluindo o Português.
 - Tok2Vec pertence à família dos embeddings: “**curtos densos dinâmicos**”.
 - É uma tecnologia proprietária da spaCy
 - <https://explosion.ai/blog/deep-learning-formula-nlp>
 - **500.000** embeddings no modelo “pt_core_news_lg” !!!
 - Fácil de usar !!!

Introdução (2/3)

- Exemplo:
 - Palavra “**yorkshire**” representada em um embedding de 300 dimensões Tok2Vec.

```
v1 = nlp.vocab['yorkshire'].vector
print(v1)
```

```
[ 0.33585   -1.5649    0.28539   -0.013775  -0.33551   -0.24744
 -0.013662  -0.5053    0.38919    0.48436   -0.6157    0.74288
 -2.2552    -0.34036    1.245    -2.0483    0.9266    -1.854
 0.89929    -0.39454   -1.2812    0.40175    0.68672   -0.79371
 0.36184    1.1702    -0.56273   -0.18     -1.2379   -0.419
 -0.41417    0.87737   -1.1372   -0.29888    1.5209    0.17526
 0.20513    0.56115   -0.65953    0.52245   -0.57426   -0.84597
 0.41822    -0.080914    0.88876   -0.23958   -0.88278   -0.64046
 0.65193    -0.3397    1.156    0.29984    1.0211   -0.29732
 -0.35304    1.0972    -0.98051   -0.056678   -0.91302    1.0922
 -1.6067    0.4631    0.34518    0.55791   -1.326    -0.5671
 -0.32722   -0.69297    0.77295    0.88348    1.7332   -0.37246
 -0.21374   -0.54183   -0.8945   -0.58988   -0.18005   -0.61715
 -0.62003   -0.46288    1.4362   -0.077357    0.77405    0.96459
 -0.80276    0.5546   -0.06119   -2.0968   -0.20403    0.2855
 -1.1182   -0.69802    0.42064    0.10657    1.5698    0.32313
 0.97309    0.16609    1.9889   -0.39007    1.3761   -0.73659
 -0.53698    0.19517   -1.6457   -0.38943   -0.10203    1.1841
 -0.14901    0.56836   -0.48705   -2.2216   -0.65739    0.37724
 -0.55766    1.0244   -0.29677   -1.8196    0.88955   -0.48285
 1.0507    -1.1261    0.35113   -1.0326    0.43839   -1.3328
 0.77678    0.99464   -1.441   -1.1952   -0.54517   -1.2298
 -0.004037   -0.5351    0.85815   -0.27054    1.2091    1.7944
 -0.29597   -0.81947   -0.99808   -2.2094    0.60518    0.18788
 -0.76305    1.6368    0.54882   -0.14999   -1.0127    0.22344
 0.41953    0.26247    0.39449   -1.3413    0.39896   -0.95934
 -0.54772   -0.12659    1.3219   -0.85647    0.98892   -0.831
 -1.2271    0.14064   -0.68062   -0.65595    0.34786    0.088964
 0.1739    0.41134   -0.31728   -0.45795    0.31028    0.12206
 -0.65003    0.34389    1.873    1.0868   -1.8812    0.7658
 -2.025    0.084462    0.64524   -0.84264    2.0689   -0.76849
 -0.1331   -1.7494   -1.2231    1.1067   -1.2282   -1.0043
 0.56094   -1.2305   -0.62509   -1.9044   -1.4847   -0.24958
 0.28698    0.61108    0.40653   -0.69055   -0.20862   -0.5237
 -0.98617   -0.0083708    0.68752    0.51341    0.88271   -0.74849
 -1.0651    0.16491   -0.73019    0.20947    1.1388    2.9898
 -0.71106   -0.28291   -0.95785   -1.154    0.77568    1.4343
 -0.14754    0.57098   -0.70668    0.019001   -0.67907    0.032678
 1.0617    0.062576    0.61975    0.57988    1.6049    0.25849
 0.63842    0.34488    0.66492   -0.25549    0.71643    0.088011
 0.17994   -0.90814   -1.3435   -0.040281    0.18399   -0.99581
 -1.6741   -1.359    0.22893   -1.1433   -0.12425    0.24456
 0.51337    0.20649   -0.29187    1.0329   -0.095624    1.1894
 -0.49944   -0.067464    0.43022   -0.7271   -1.2456   -0.069048
 -0.32117    0.93746    0.84282    0.41096   -0.3927    1.0984
 0.46101   -0.39119    0.76457   -0.12572   -0.16426   -0.4845
 -0.76046    0.88486   -0.75781    1.1995    0.47471    0.32066
 -0.2531   -0.48383   -0.092128    0.38621   -0.066206    0.53133
 -0.84042   -0.060871    0.08303    0.68859    1.0575    0.88875
 1.2956    1.0594    0.22777   -0.35526    0.078052    0.062136 ]
```

Introdução (3/3)

- **Similaridade entre Embeddings**
 - Embeddings similares devem, por princípio, representar palavras ou textos similares.
 - Sendo assim, é importante encontrar alguma maneira confiável de medir a similaridade entre embeddings.
 - Nos próximos slides veremos como isso é feito na prática.

Similaridade entre Embeddings (1/8)

- Como medir a similaridade entre 2 embeddings v_1 e v_2 ?
 - Normalmente, utiliza-se a medida do **cosseno** do ângulo entre v_1 e v_2
 - Como veremos nos próximos slides, o cosseno é **baseado no produto escalar** dos vetores, algo muito comum na PLN.
- **Produto Escalar** (*dot product* ou *inner product*):
 - Dados dois vetores de n posições v_1 e v_2 , o produto escalar $v_1 \cdot v_2$ entre eles é dado por:

$$v_1 \cdot v_2 = v_{1[1]}v_{2[1]} + v_{1[2]}v_{2[2]} + \dots + v_{1[n]}v_{2[n]}$$

Similaridade entre Embeddings (2/8)

- Por que o Produto Escalar?
 - Ele atua como uma medida de similaridade porque tende a ser alto apenas quando v_1 e v_2 têm valores grandes nas mesmas posições.

$$v_1 \cdot v_2 = v_{1[1]}v_{2[1]} + v_{1[2]}v_{2[2]} + \dots + v_{1[n]}v_{2[n]}$$

- Além disso, **vetores ortogonais** (com zeros em dimensões diferentes), terão um produto escalar de 0.
 - Isso representa bem a forte dissimilaridade entre eles.
- Entretanto, o produto escalar possui alguns problemas...

Similaridade entre Embeddings (3/8)

- **Problema 1**: o Produto Escalar viola a definição de função de similaridade
 - Conforme visto em aulas anteriores, **função de similaridade** entre esses dois objetos s e t , uma é uma função $S : (s, t) \rightarrow [0;1]$ que satisfaz **três propriedades**:
 1. $S(s, t) = 1$ se $s = t$;
 2. $S(s, t) \approx 1$ quando s é muito parecido com t ;
 3. $S(s, t) \approx 0$ quando s é muito diferente de t .
 - Entretanto, o produto escalar pode ser um valor entre $-\infty$ e $+\infty$.

Similaridade entre Embeddings (4/8)

- **Problema 2:** o Produto Escalar favorece vetores com muitos valores grandes.
 - Embeddings esparsos (não é o caso do Tok2Vec) de **palavras mais frequentes**, como “fui” ou “você” tendem a possuir **valores grandes** em muitas posições...
 - ... isso porque essas palavras tendem a coocorrer com diversas outras.

fui	150	50	...	480	235	42
-----	-----	----	-----	-----	-----	----

você	302	89	...	240	389	361
------	-----	----	-----	-----	-----	-----

- Por outro lado, embeddings de **palavras infrequentes**, como “vós” tendem a possuir **zero** ou **valores pequenos** em muitas posições...
 - ... isso porque essas palavras coocorrem com poucas outras

vós	0	2	...	0	0	1
-----	---	---	-----	---	---	---

Similaridade entre Embeddings (5/8)

- **Problema 2:** o Produto Escalar favorece vetores com muitos valores grandes.
 - Dessa forma, “fui” e “você” terão produto escalar com valor superior a “vós” e “você”
 - Embora “vós” e “você” sejam palavras mais similares

fui	150	50	...	480	235	42
você	302	89	...	240	389	361
vós	0	2	...	0	0	1

- Isso é um **problema**!
 - Queremos uma medida de similaridade que funcione bem para palavras frequentes e infrequentes e para embeddings esparsos ou densos.
 - *É aí que o cosseno entra para resolver !!!!*

Similaridade entre Embeddings (6/8)

- **Cosseno**

- Vetores de palavras frequentes (com valores grandes em mais posições) possuem **comprimento** (ou **norma** ou **módulo**) maior.
- A norma de um vetor **v**, representada por **$\|v\|$** é computada por:

$$\|v\| = \sqrt{v_{[1]}^2 + v_{[2]}^2 + \dots + v_{[n]}^2}$$

- Então a solução é simples: **dividir** o valor do **produto escalar pelo comprimento** de cada vetor !!!
- Isso resulta exatamente no **cosseno do ângulo** entre os vetores:

$$v_1 \cdot v_2 = \|v_1\| \cdot \|v_2\| \cdot \cos(v_1, v_2) \Rightarrow \cos(v_1, v_2) = \frac{v_1 \cdot v_2}{\|v_1\| \cdot \|v_2\|}$$

Similaridade entre Embeddings (7/8)

- **Cosseno dos Vetores**

$$\cos(v_1, v_2) = \frac{v_1 \cdot v_2}{\|v_1\| \cdot \|v_2\|}$$

- Numerador = produto escalar
- Denominador = produto das normas (comprimentos)

- **Como funciona?**

- Quanto menor o ângulo entre os vetores, mais similares eles são.
 - O que interessa é o ângulo, não o tamanho dos vetores.
 - Funciona bem para vetores densos ou esparsos.
- Para embeddings que armazenam frequências, resultará sempre em um valor entre 0 e 1.
 - $\text{Cos}(s, t) \approx 1$, se os vetores são muito similares ($\cos(0^\circ) = 1$)
 - $\text{Cos}(s, t) \approx 0$, se os vetores são muito dissimilares ($\cos(90^\circ) = 0$)

Similaridade entre Embeddings (8/8)

- **Cosseno** – o que importa é o **ângulo entre os vetores** e não a magnitude

	pie	data	computer
cherry	442	8	2
digital	5	1683	1670
information	5	3982	3325

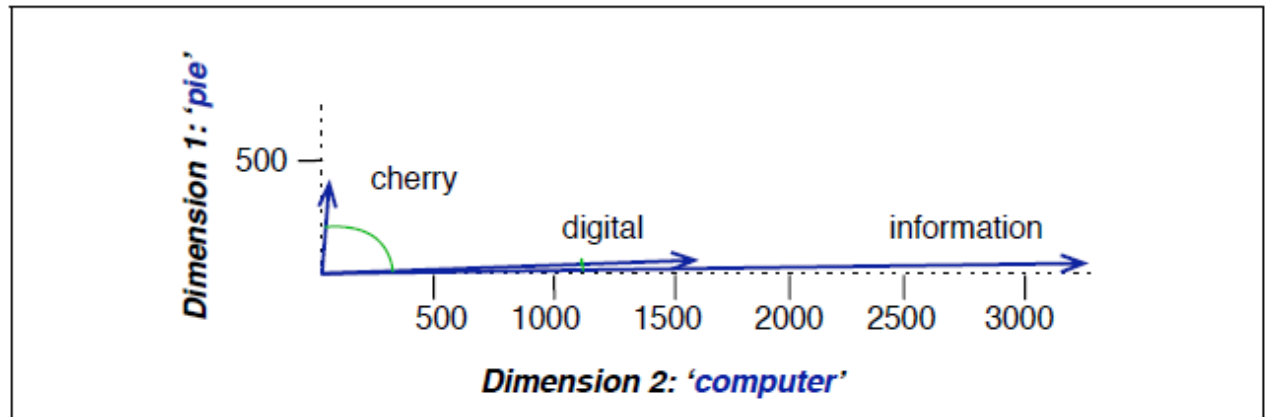


Figure 6.8 A (rough) graphical demonstration of cosine similarity, showing vectors for three words (*cherry*, *digital*, and *information*) in the two dimensional space defined by counts of the words *computer* and *pie* nearby. The figure doesn't show the cosine, but it highlights the angles; note that the angle between *digital* and *information* is smaller than the angle between *cherry* and *information*. When two vectors are more similar, the cosine is larger but the angle is smaller; the cosine has its maximum (1) when the angle between two vectors is smallest (0°); the cosine of all other angles is less than 1.

$$\cos(\text{cherry}, \text{information}) = \frac{442 * 5 + 8 * 3982 + 2 * 3325}{\sqrt{442^2 + 8^2 + 2^2} \sqrt{5^2 + 3982^2 + 3325^2}} = .018$$

$$\cos(\text{digital}, \text{information}) = \frac{5 * 5 + 1683 * 3982 + 1670 * 3325}{\sqrt{5^2 + 1683^2 + 1670^2} \sqrt{5^2 + 3982^2 + 3325^2}} = .996$$

Exemplos na Spacy

- Notebook:
 - https://colab.research.google.com/drive/12dP0rep7Ega_M5vRb5EFVB5-wlO2X01W?usp=sharing

Referências

- JURAFSKY, D.; MARTIN, J. H. Vector Semantics and Embeddings. *In*: JURAFSKY, D.; MARTIN, J. H. **Speech and Language Processing**. 3. ed. *draft* [s.l.] : [s.n.], 2025. cap 6, p. 102-117. Disponível em: <https://web.stanford.edu/~jurafsky/slp3/>. Acesso em: 28 mar. 2025.